

FLOWTER

Documentação Técnica do Software com Modelagem UML

Sistema de Gestão Escolar e Geração de Horários

Campo	Descrição
Projeto	Flowter - Sistema de Gestão Escolar
Tipo	Aplicação web
Tecnologias	Python, Flask, MySQL/MariaDB, HTML, CSS e JavaScript
Área de negócio	Gestão escolar, organização de horários e acompanhamento pedagógico
Versão da documentação	1.0 - Revisada e aprimorada
Data	08/06/2026

Esta documentação reorganiza o material existente, confere as entidades do diagrama de banco enviado e transforma o modelo em uma visão UML mais limpa, legível e pronta para evolução do projeto.

Sumário executivo

- O Flowter centraliza cadastros escolares, organiza turmas, professores, disciplinas e permite montar grades horárias com validação de conflitos.
- O modelo atual possui 14 entidades principais, incluindo usuários, escolas, turmas, disciplinas, professores, aulas oficiais, horários temporários, relatórios e prontuários de alunos.
- A documentação foi aprimorada para separar visão de negócio, requisitos, regras, dados, UML e recomendações técnicas.
- A arquitetura sugerida mantém o padrão em camadas: rotas, modelos, banco de dados, serviços, exportações, utilitários e interface web.

1. Visão geral do software

O Flowter é uma aplicação web voltada para escolas que precisam transformar a montagem de horários em um processo mais seguro, rastreável e inteligente. A proposta é reduzir conflitos manuais, organizar vínculos entre professores, turmas e disciplinas, e oferecer uma base administrativa para relatórios e prontuários.

Item	Definição
Problema central	Montagem manual de horários com risco de choque de professor, turma, disciplina, período e disponibilidade.
Solução proposta	Sistema web com cadastros, regras, geração de grade, ajustes manuais, exportações e módulos de acompanhamento.
Usuários principais	Administrador, coordenador escolar e funcionário.
Unidade de trabalho	Escola, turno, turma, disciplina, professor, aula e período.
Resultado esperado	Grade organizada, consultável, exportável e com menor probabilidade de conflito.

2. Termo de Abertura do Projeto - TAP

2.1 Justificativa

Instituições escolares precisam organizar horários de forma consistente. Quando a gestão é manual, aparecem conflitos como professor alocado em duas turmas no mesmo período, excesso de aulas em sequência, falta de controle de disponibilidade e baixa rastreabilidade das alterações. O Flowter nasce para transformar a grade em um mapa vivo: cada aula no lugar certo, cada vínculo documentado e cada decisão com lastro técnico.

2.2 Objetivo geral

Desenvolver uma aplicação web para auxiliar escolas na gestão de usuários, escolas, turmas, disciplinas, professores, cargas horárias, horários oficiais, horários temporários, relatórios de professores e prontuários de alunos.

2.3 Objetivos específicos

- Permitir cadastro, autenticação, verificação de e-mail e recuperação de senha.
- Controlar perfis de acesso e vínculos entre usuários e escolas.
- Gerenciar turmas, disciplinas e professores por escola e turno.
- Registrar cargas horárias por professor, turma e disciplina.
- Gerar, salvar, mover e limpar aulas da grade oficial.
- Criar horários temporários sem alterar a grade oficial.
- Registrar relatórios de professores e prontuários de alunos.

- Exportar horários e relatórios em PDF e Excel.
- Manter backups ocultos de escolas para restauração administrativa.

2.4 Escopo

Dentro do escopo	Fora do escopo inicial
Aplicação web com autenticação e sessão	Aplicativo mobile nativo
CRUD de escolas, turmas, disciplinas, professores e usuários	Matrícula completa de alunos
Geração e manutenção de grade horária	Controle financeiro escolar
Relatórios, prontuários e exportações	Integração com diário eletrônico externo
Administração de vínculos e backups	Integração com APIs governamentais

3. Atores e perfis de acesso

Ator	Responsabilidade	Permissões esperadas
Administrador	Governa o sistema, usuários, escolas e backups.	Acesso total, criação de vínculos, restauração e gestão de perfis.
Coordenador	Opera a escola vinculada e organiza a grade.	CRUD escolar, geração de horários, relatórios e prontuários.
Funcionário	Consulta e exporta informações autorizadas.	Visualização, exportação e acompanhamento conforme vínculo.
Sistema Gerador de Grade	Aplica regras e sugere alocações.	Executa validações de conflito e grava sugestões/aulas.
Equipe escolar	Consome horários e relatórios.	Consulta indireta por exportações e telas autorizadas.

4. Requisitos funcionais

Código	Requisito
RF01	Permitir cadastro e login de usuários.
RF02	Exigir verificação de e-mail antes do acesso.
RF03	Permitir recuperação e redefinição de senha.
RF04	Controlar permissões por perfil.
RF05	Cadastrar, editar e excluir escolas.
RF06	Cadastrar turmas por escola e turno.
RF07	Cadastrar disciplinas por escola e turno.
RF08	Cadastrar professores, disponibilidade e cargas horárias.
RF09	Gerar horários escolares oficiais.
RF10	Validar conflitos de professor, turma, disciplina, dia e período.
RF11	Permitir movimentação manual de aulas.
RF12	Permitir horários temporários por intervalo de datas.
RF13	Registrar relatórios de professores.
RF14	Registrar prontuários de alunos com prioridade, status e feedback.
RF15	Exportar horários e relatórios em PDF e Excel.
RF16	Administrar usuários, vínculos e backups.
RF17	Permitir escolas ocultas como cópias de segurança administrativa.
RF18	Registrar rastreabilidade de criação, exclusão e feedback quando aplicável.

5. Requisitos não funcionais

Código	Requisito
RNF01	Executar em ambiente web com Flask.
RNF02	Utilizar MySQL 8 ou MariaDB 10 ou superior.
RNF03	Armazenar senhas somente com hash.
RNF04	Usar sessão segura com cookies HTTPOnly.
RNF05	Aplicar proteção CSRF em requisições mutáveis.

RNF06	Permitir configuração por variáveis de ambiente.
RNF07	Separar responsabilidades entre rotas, modelos, banco, exportações e serviços.
RNF08	Gerar arquivos temporários de exportação com controle de ciclo de vida.
RNF09	Manter índices e restrições para evitar duplicidades críticas.
RNF10	Registrar data de criação e exclusão lógica nos módulos sensíveis.

6. Regras de negócio principais

1. Uma escola possui turmas, disciplinas, professores, aulas, horários temporários, relatórios e prontuários.
2. Um usuário pode possuir ou acessar múltiplas escolas por meio da tabela de vínculo.
3. Uma turma pertence a uma escola e a um turno, e define a quantidade de aulas por dia.
4. Uma disciplina pertence a uma escola e a um turno, podendo ter cor para exibição visual.
5. Um professor pertence a uma escola e turno, possui disponibilidade e limite máximo de aulas semanais.
6. A carga horária liga professor, turma e disciplina, definindo quantas aulas devem ocorrer por semana.
7. Uma aula oficial deve pertencer a escola, turno, turma, professor, disciplina, dia e período.
8. Duas aulas não devem colocar o mesmo professor no mesmo dia e período dentro do mesmo turno.
9. Uma turma não deve possuir duas aulas simultâneas no mesmo dia e período.
10. Horários temporários valem por intervalo de datas e não substituem definitivamente a grade oficial.
11. Relatórios de professores e prontuários de alunos devem preservar histórico por criação, feedback e exclusão lógica.

7. Modelo de dados conferido no diagrama

O diagrama enviado apresenta as entidades abaixo. Os nomes truncados visualmente foram normalizados nesta documentação para facilitar leitura e manutenção.

Entidade	Papel no domínio
usuarios	Usuários do sistema, autenticação, perfil e controle de login.
escolas	Unidades escolares, dono/vínculo, ocultação e backup.
usuarios_escolas	Tabela associativa entre usuários e escolas.
turmas	Turmas por escola, turno e quantidade de aulas por dia.
disciplinas	Disciplinas por escola e turno, com cor de exibição.
professores	Professores por escola e turno, com disponibilidade e limite semanal.
professores_disciplinas	Associação N:N entre professores e disciplinas.
professores_turmas	Associação N:N entre professores e turmas.
professores_cargas	Carga horária planejada por professor, turma e disciplina.
aulas	Grade oficial salva no sistema.
grade_sugestao	Sugestões geradas antes de confirmar ou revisar a grade.
horarios_temporarios	Eventos/aulas temporárias por período de datas.
relatorios_professores	Ocorrências, faltas e registros de professores.
prontuarios_alunos	Registros de alunos, prioridades, status e feedback.

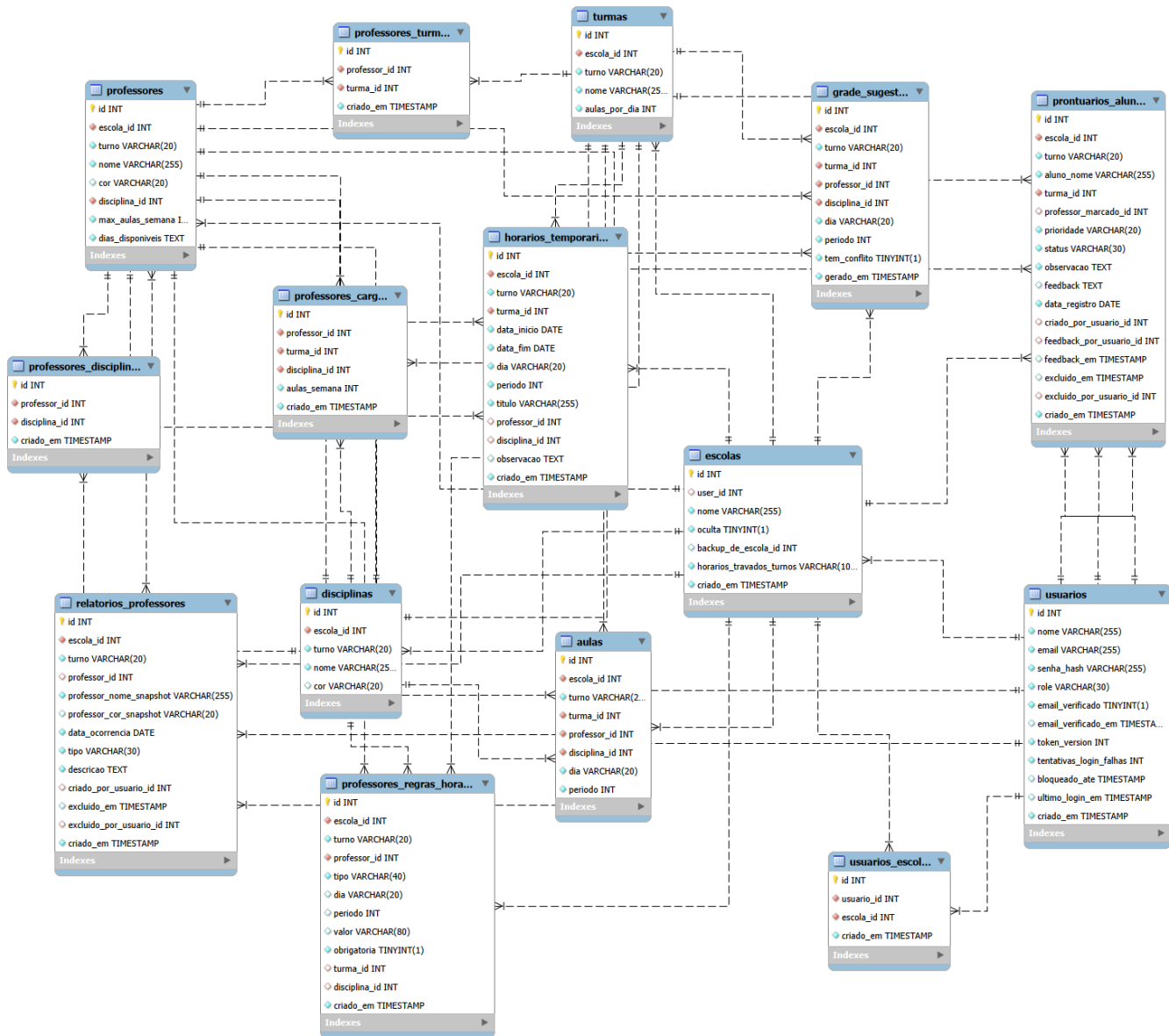
8. Dicionário de dados resumido

Tabela	Campos principais
usuarios	id, nome, email, senha_hash, role, email_verificado, email_verificado_em, token_version, tentativas_login_falhas, bloqueado_ate, ultimo_login_em, criado_em
escolas	id, user_id, nome, ocuta, backup_de_escola_id, horarios_travados_turnos, criado_em
turmas	id, escola_id, turno, nome, aulas_por_dia
disciplinas	id, escola_id, turno, nome, cor

professores	id, escola_id, turno, nome, cor, disciplina_id, max_aulas_semana, dias_disponiveis
professores_cargas	id, professor_id, turma_id, disciplina_id, aulas_semana, criado_em
aulas	id, escola_id, turno, turma_id, professor_id, disciplina_id, dia, periodo
grade_sugestao	id, escola_id, turno, turma_id, professor_id, disciplina_id, dia, periodo, tem_conflito, gerado_em
horarios_temporarios	id, escola_id, turno, turma_id, data_inicio, data_fim, dia, periodo, titulo, professor_id, disciplina_id, observacao, criado_em
relatorios_professores	id, escola_id, turno, professor_id, professor_nome_snapshot, professor_cor_snapshot, data_ocorrencia, tipo, descricao, criado_por_usuario_id, excluido_em, excluido_por_usuario_id, criado_em
prontuarios_alunos	id, escola_id, turno, aluno_nome, turma_id, professor_marcado_id, prioridade, status, observacao, feedback, data_registro, criado_por_usuario_id, feedback_por_usuario_id, feedback_em, excluido_em, excluido_por_usuario_id, criado_em

9. Diagrama original conferido

A imagem abaixo foi usada como base para conferir entidades, campos e relacionamentos principais. Para visualização ampliada, use também o arquivo SVG/PDF original enviado.



10. UML - Diagrama de classes conceitual

Abaixo está a versão UML conceitual revisada em Mermaid. Ela separa entidades centrais, associações N:N e módulos de registro, evitando que o diagrama fique poluído como o modelo físico completo.

```
classDiagram
class Usuario {
+int id
+string nome
+string email
+string senha_hash
+string role
+bool email_verificado
+datetime ultimo_login_em
}
class Escola {
+int id
+int user_id
+string nome
}
```

```

+bool oculta
+int backup_de_escola_id
+string horarios_travados_turnos
}
class UsuarioEscola {
+int id
+int usuario_id
+int escola_id
+datetime criado_em
}
class Turma {
+int id
+int escola_id
+string turno
+string nome
+int aulas_por_dia
}
class Disciplina {
+int id
+int escola_id
+string turno
+string nome
+string cor
}
class Professor {
+int id
+int escola_id
+string turno
+string nome
+string cor
+int max_aulas_semana
+text dias_disponiveis
}
class ProfessorDisciplina
class ProfessorTurma
class ProfessorCarga {
+int aulas_semana
}
class Aula {
+string dia
+int periodo
}
class GradeSugestao {
+bool tem_conflito
+datetime gerado_em
}
class HorarioTemporario {
+date data_inicio
+date data_fim
+string titulo
+text observacao
}
class RelatorioProfessor {
+date data_ocorrencia
+string tipo
+text descricao
}
class ProntuarioAluno {
+string aluno_nome
+string prioridade
+string status
+text feedback
}

```

```

Usuario "1" --> "0..*" Escola : cria
Usuario "1" --> "0..*" UsuarioEscola : acessa
Escola "1" --> "0..*" UsuarioEscola : vincula
Escola "1" --> "0..*" Turma
Escola "1" --> "0..*" Disciplina
Escola "1" --> "0..*" Professor
Professor "1" --> "0..*" ProfessorDisciplina
Disciplina "1" --> "0..*" ProfessorDisciplina
Professor "1" --> "0..*" ProfessorTurma
Turma "1" --> "0..*" ProfessorTurma
Professor "1" --> "0..*" ProfessorCarga
Turma "1" --> "0..*" ProfessorCarga
Disciplina "1" --> "0..*" ProfessorCarga
Turma "1" --> "0..*" Aula
Professor "1" --> "0..*" Aula
Disciplina "1" --> "0..*" Aula
Aula <.. GradeSugestao : pode originar
Turma "1" --> "0..*" HorarioTemporario
Professor "0..1" --> "0..*" HorarioTemporario
Disciplina "0..1" --> "0..*" HorarioTemporario
Professor "0..1" --> "0..*" RelatorioProfessor
Turma "1" --> "0..*" ProntuarioAluno
Professor "0..1" --> "0..*" ProntuarioAluno

```

11. UML - Casos de uso

Modelo textual em Mermaid para renderização no GitHub, VS Code ou Mermaid Live Editor.

flowchart LR

```

Admin[Administrador]
Coord[Coordenador]
Func[Funcionário]
Sistema[Gerador de Grade]

```

```

UC1((Gerenciar usuários))
UC2((Gerenciar escolas))
UC3((Vincular usuários a escolas))
UC4((Cadastrar turmas))
UC5((Cadastrar disciplinas))
UC6((Cadastrar professores))
UC7((Definir cargas horárias))
UC8((Gerar grade horária))
UC9((Mover aula manualmente))
UC10((Criar horário temporário))
UC11((Registrar relatório de professor))
UC12((Registrar prontuário de aluno))
UC13((Exportar PDF/Excel))

```

```

Admin --> UC1
Admin --> UC2
Admin --> UC3
Coord --> UC4
Coord --> UC5
Coord --> UC6
Coord --> UC7
Coord --> UC8
Coord --> UC9
Coord --> UC10
Coord --> UC11
Coord --> UC12

```


Coord --> UC13
Func --> UC13
Func --> UC12
Sistema --> UC8

12. UML - Sequência: geração de grade horária

sequenceDiagram

actor Coordenador
participant Web as Interface Web
participant Auth as Controle de Acesso
participant Scheduler as Gerador de Grade
participant Validador as Validador de Conflitos
participant DB as Banco de Dados

Coordenador->>Web: Solicita geração da grade
Web->>Auth: Verifica perfil e vínculo com a escola
Auth->>Web: Acesso autorizado
Web->>DB: Carrega turmas, professores, disciplinas e cargas
DB->>Web: Dados de configuração
Web->>Scheduler: Gerar sugestões
Scheduler->>Validador: Validar professor/turma/dia/período
Validador->>Scheduler: Conflitos encontrados ou grade válida
Scheduler->>DB: Salva grade_sugestao
DB->>Web: Resultado da geração
Web->>Coordenador: Exibe sugestões e conflitos

13. UML - Atividade: cadastro até grade oficial

flowchart TD

A[Início] --> B[Cadastrar escola]
B --> C[Cadastrar turmas]
C --> D[Cadastrar disciplinas]
D --> E[Cadastrar professores]
E --> F[Vincular professor a disciplina e turma]
F --> G[Definir cargas horárias]
G --> H[Gerar grade sugestão]
H --> I{Existem conflitos?}
I -- Sim --> J[Ajustar regras, disponibilidade ou carga]
J --> H
I -- Não --> K[Salvar grade oficial em aulas]
K --> L[Exportar ou consultar horários]
L --> M[Fim]

14. Arquitetura lógica recomendada

Camada/Módulo	Responsabilidade
src/app.py	Inicialização do Flask, blueprints, segurança e criação/migração de tabelas.
src/routes/	Rotas HTTP, formulários, sessão, templates e orquestração de casos de uso.
src/models/	Regras de negócio e acesso a dados das entidades principais.
src/database/	Conexão, schema, migrações simples e scripts de inicialização.
src/services/	Serviços transversais: autenticação, e-mail, grade, permissões e backups.
src/utils/	Funções auxiliares, validações e checagem de conflitos.
src/exports/	Geração de PDF e Excel.

src/templates/ e src/static/	Interface web, componentes visuais, CSS e JavaScript.
tests/	Testes automatizados de regras, rotas e persistência.

15. Recomendações técnicas após conferência

Ponto	Melhoria recomendada
Padronizar nomes	Usar nomes completos: professores_cargas, professores_turmas, professores_disciplinas, horarios_temporarios e prontuarios_alunos.
Criar restrições únicas	Evitar duplicidade em usuarios.email, usuarios_escolas(usuario_id, escola_id), turmas(escola_id, turno, nome), disciplinas(escola_id, turno, nome) e aulas(escola_id, turno, turma_id, dia, periodo).
Rever duplicidade de disciplina do professor	A tabela professores possui disciplina_id e também existe professores_disciplinas. Definir se disciplina_id é disciplina principal ou remover para evitar inconsistência.
Formalizar turno	Transformar turno em enum validado ou tabela de domínio para impedir variações como Matutino/manhã/M.
Auditare regras de conflito	Centralizar as regras em um serviço único para geração automática e movimentação manual.
Usar soft delete onde houver histórico	Relatórios e prontuários já possuem exclusão lógica. Avaliar o mesmo para escolas, turmas e professores se houver necessidade de auditoria.
Adicionar timestamps consistentes	Padronizar criado_em, atualizado_em e excluido_em onde fizer sentido.
Documentar cascatas	Definir ON DELETE/ON UPDATE para impedir dados órfãos, especialmente em aulas, vínculos e cargas.
Separar sugestão de oficial	Manter grade_sugestao como rascunho e aulas como grade oficial confirmada.
Backups de escola	Documentar claramente quando backup_de_escola_id aponta para escola original ou cópia oculta.

16. Índices e restrições sugeridas

Tabela	Índices/restrições sugeridos
usuarios	UNIQUE(email); INDEX(role); INDEX(email_verificado).
usuarios_escolas	UNIQUE(usuario_id, escola_id).
turmas	UNIQUE(escola_id, turno, nome).
disciplinas	UNIQUE(escola_id, turno, nome).
professores	INDEX(escola_id, turno, nome); INDEX(disciplina_id).
professores_cargas	UNIQUE(professor_id, turma_id, disciplina_id).
aulas	UNIQUE(escola_id, turno, turma_id, dia, periodo); INDEX(professor_id, dia, periodo).
grade_sugestao	INDEX(escola_id, turno, gerado_em); INDEX(tem_conflito).
horarios_temporarios	INDEX(escola_id, turno, data_inicio, data_fim); INDEX(turma_id, dia, periodo).
relatorios_professores	INDEX(escola_id, turno, professor_id, data_ocorrencia).
prontuarios_alunos	INDEX(escola_id, turno, turma_id, status, prioridade).

17. Critérios de aceitação

- O usuário consegue se cadastrar, verificar e-mail, autenticar e recuperar senha.
- O administrador consegue criar usuários, escolas, vínculos e backups.
- O coordenador consegue gerenciar turmas, disciplinas, professores, cargas e horários.
- O funcionário consegue consultar e exportar informações autorizadas.
- O sistema impede professor duplicado no mesmo dia/período/turno.
- O sistema impede turma com duas aulas no mesmo dia/período.
- A grade sugestão mostra conflitos antes da confirmação.

- PDF e Excel são gerados com dados corretos.
- Relatórios e prontuários mantêm rastreabilidade de criação, feedback e exclusão.

18. Roadmap de evolução

Fase	Evolução sugerida
1 - Consolidação	Corrigir nomes, restrições, índices e documentação do schema.
2 - Qualidade	Criar testes automatizados para conflitos, permissões e geração de grade.
3 - Usabilidade	Melhorar tela de montagem manual com arrastar e soltar e alertas visuais.
4 - Inteligência	Adicionar heurísticas mais fortes para distribuir aulas e reduzir sequências ruins.
5 - Produto	Preparar versão SaaS multi-escola com assinatura, auditoria e painel administrativo avançado.

19. Conclusão

O modelo atual do Flowter já possui uma base consistente para um sistema real de gestão de horários. O próximo salto é transformar o banco em uma arquitetura mais governada: nomes claros, restrições fortes, regras centralizadas e documentação viva. Assim, o software deixa de ser apenas um conjunto de tabelas e passa a ser um produto com direção, lastro técnico e potencial de escala.